HW6

Sri Srinivasan

August 2025

Problem 1

(a)

The output of the j-th node at layer k is the j-th entry of the vector $A^k x$, where x is the initial input vector. Thus, if node i has initial input $x = e_i$, then the j-th node at layer k outputs:

$$(A^k)_{j,i}$$

(b)

By definition of $L_k(i, j)$ as the number of paths of length k from i to j, and from the hint that $(A^k)_{j,i} = L_k(i, j)$, the j-th output of node is at layer k is $\mathrm{L}_k(i, j)$.

(c)

Since the operation at each layer is a permutation-invariant sum over neighbors, the update function just computes

$$\text{output}_j^{(k)} = \sum_i A_{ji} \cdot \text{output}_i^{(k-1)}$$

The update function is: sum the weighted inputs from neighbors. For all j:

$$x_j^{(k)} = \sum_{i \in \mathcal{N}(j)} A_{ji} x_i^{(k-1)}$$

(d)

If we use max aggregation instead of sum, the output at each node j would be

$$x_j^{(k)} = \max_{i \in \mathcal{N}(j)} \{A_{ji} x_i^{(k-1)}\}$$

Problem 2

(a)

(i)

$$\xi_i^\ell = x_i^{\ell-1} + w_1 \sum_{j=1}^{m_i} x_j^{\ell-1}$$

(ii)

$$\xi_i^\ell = \max\left(x_i^{\ell-1}, w_1 x_1^{\ell-1}, w_2 x_2^{\ell-1}, \ldots, w_{m_i} x_{m_i}^{\ell-1}\right)$$

(iii)

$$\xi_i^\ell = \max\left(x_i^{\ell-1}, w_1 x_1^{\ell-1}, w_2 x_2^{\ell-1}, \ldots, w_m x_{m_i}^{\ell-1}\right)$$

All three functions are valid for aggregation.

(b)

Given y_i (true labels) and $\hat{y}_i$ (predicted probabilities), the binary cross-entropy loss over the training points is:

$$\frac{1}{n} \sum_{i \in T} \left(y_i \log \frac{1}{\hat{y}_i} + (1 - y_i) \log \frac{1}{1 - \hat{y}_i} \right)$$

where T is the set of training points.

(c)

Given update rule:

$$s_i^\ell = s_i^{\ell-1} + W_2 \cdot \frac{1}{n_i} \sum_{j=1}^{n_i} \tanh(W_1 x_j^{\ell-1})$$

(i)

Residual connection

(ii)

Each $\tanh(W_1 x_j^{\ell-1})$ has shape $k \times 1 \rightarrow$ average is $k \times 1 \rightarrow W_2$ must be $d \times k$

(iii)

We have

$$s_i^\ell = s_i^{\ell-1} + W_2 \cdot \frac{1}{n_i} \sum_{j=1}^{n_i} \tanh(W_1 x_j^{\ell-1})$$

With identity as message (no W_1, just $x_j^{\ell-1}$), this becomes

$$s_i^{\ell} = s_i^{\ell-1} + W_2 \cdot \frac{1}{n_i} \sum_{j=1}^{n_i} x_j^{\ell-1}$$

This corresponds to the average neighbor state, which corresponds to residual add.

Problem 4

A

Each atom is represented as a node. Each bond is represented as an edge connecting two nodes. The graph is undirected since bonds are mutual. Node features include atomic number, weight, element type, and molecular context. Edge features could include bond length and bond type. A global feature might encode the overall molecule type. The final GNN layer outputs edge embeddings. We can apply a classifier or a scoring function to each edge representation to predict the likelihood that a bond breaks under heat. Each node is represented using a feature vector containing atom-specific information like atomic number, weight, valency, electronegativity, and type of molecule context.

B

CNN	**GNN**
Image classification	Graph-level prediction problem Node-level prediction problem
Color jitter data augmentation	Random feature perturbation of node/edge features
Image flip data augmentation	Random permutation or isomorphism of graph structure
Channel dropout	Node/edge feature dropout
Zero padding edges	Padding with dummy nodes/edges
ResNet skip connections	Residual connections in GNN layers
Blurring an image	Predicting missing values of nodes Edge-order invariance (neighbors unordered)

C

Use a GNN that aggregates over neighbors (using mean, sum, or max pooling) so that even if some nodes lack features, their neighbors contribute enough context to make predictions.

D

Doubling the number of nodes increases the number of node feature vectors, but the number of learnable parameters remains constant if the model uses shared

weights across all nodes. The computational cost increases, but not the number of parameters.

Weights are learned in the update functions f_v, f_e, and f_u that are applied during the message passing and pooling steps.

Encode direction using edge attributes, or use separate message functions for incoming and outgoing edges.

MLP-like GNN: Use a fixed number of GNN layers and apply the same learned weights in each. RNN-like GNN: Share weights across layers and feed the graph through multiple timesteps, using hidden states (like in GRUs or LSTMs) to propagate information.

E

Each person is a node, and edges represent social interactions. Since we are predicting labels for half the nodes, this is a semi-supervised node-level prediction task. Use a GNN that propagates features across the network. The model generalizes to predict for unlabeled nodes.

Problem 5

a

Let $\mathbf{w}^*$ be the minimum-norm solution to $X\mathbf{w} = \mathbf{y}$. Since X has full row rank, one such solution is

$$\mathbf{w}^* = X^\top (XX^\top)^{-1}\mathbf{y}.$$

Define new coordinates $\mathbf{w}' = \mathbf{w} - \mathbf{w}^*$. Then:

$$X\mathbf{w}' = X(\mathbf{w} - \mathbf{w}^*) = X\mathbf{w} - X\mathbf{w}^* = \mathbf{y} - \mathbf{y} = 0.$$

So the new system becomes

$$X\mathbf{w}' = 0.$$

The initial condition is $\mathbf{w}_0 = 0 \Rightarrow \mathbf{w}'_0 = -\mathbf{w}^*$.

b

Since $X \in \mathbb{R}^{n\times d}$ is full row-rank and wide ($d > n$), we can perform SVD and choose an orthonormal transformation $V \in \mathbb{R}^{d\times d}$ such that

$$\mathbf{w}' = V\mathbf{w}'' \Rightarrow XV\mathbf{w}'' = 0.$$

Let us write this as

$$[\tilde{X} \mid 0_{n\times(d-n)}]\mathbf{w}'' = 0.$$

This implies the last $d-n$ entries of $\mathbf{w}''_0$ are equal to the corresponding entries of $V^\top \mathbf{w}'_0 = -V^\top \mathbf{w}^*$. But since $\mathbf{w}^* \in \text{Row}(X^\top)$, its projection into the nullspace

of X is zero. Hence:

$$\mathbf{w}_0'' = -V^\top \mathbf{w}^* = \begin{bmatrix} * \\ \vdots \\ * \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

So the final $d - n$ entries are zeros.

c

From the above, we can now focus on solving

$$\tilde{X}\tilde{\mathbf{w}} = 0,$$

where $\tilde{\mathbf{w}} \in \mathbb{R}^n$. Each row of this system corresponds to the original row equation from $X\mathbf{w} = \mathbf{y}$ after projection. Hence, each of the n rows of $\tilde{X}$ can be obtained from the same indexed row of X.

d

At iteration $t + 1$, SGD updates

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla \mathcal{L}_{I_t}(\mathbf{w}_t),$$

where $\mathcal{L}_i(\mathbf{w}) = (y[i] - x_i^\top \mathbf{w})^2$. Since $y = X\mathbf{w}^*$, $\nabla \mathcal{L}_i = -2(y[i] - x_i^\top \mathbf{w})x_i$. In $\tilde{\mathbf{w}}$-coordinates, update becomes

$$\tilde{\mathbf{w}}_{t+1} = \tilde{\mathbf{w}}_t - 2\eta \tilde{x}_{I_t} \tilde{x}_{I_t}^\top \tilde{\mathbf{w}}_t.$$

e

Assume

$$E[\mathcal{L}(\tilde{\mathbf{w}}_{t+1}) | \tilde{\mathbf{w}}_t] < (1 - \rho)\mathcal{L}(\tilde{\mathbf{w}}_t).$$

Then exponential convergence holds. For all $\varepsilon > 0, \delta > 0$, exists T s.t.

$$P(\mathcal{L}(\tilde{\mathbf{w}}_T) < \varepsilon) \geq 1 - \delta.$$

f

$\mathcal{L}(\tilde{\mathbf{w}}) \geq 0$ and equals zero iff $\tilde{\mathbf{w}} = 0$, since $\tilde{X}$ is full rank.

g

Let

$$\mathcal{L}(\tilde{\mathbf{w}}_{t+1}) = \mathcal{L}(\tilde{\mathbf{w}}_t) + A + B$$

where

$$A = 2\tilde{\mathbf{w}}_t^\top \tilde{X}^\top \tilde{X}(\tilde{\mathbf{w}}_{t+1} - \tilde{\mathbf{w}}_t),$$
$$B = (\tilde{\mathbf{w}}_{t+1} - \tilde{\mathbf{w}}_t)^\top \tilde{X}^\top \tilde{X}(\tilde{\mathbf{w}}_{t+1} - \tilde{\mathbf{w}}_t).$$

h

For the contraction term,

$$E[A|\tilde{\mathbf{w}}_t] \leq -c_1\eta\mathcal{L}(\tilde{\mathbf{w}}_t),$$

where $c_1 > 0$ depends on the smallest singular value of $\tilde{X}$.

i

For the quadratic term,

$$E[B|\tilde{\mathbf{w}}_t] \leq c_2\eta^2\mathcal{L}(\tilde{\mathbf{w}}_t),$$

where $c_2 > 0$ depends on the largest row norm of $\tilde{X}$.

j

Putting both together,

$$E[\mathcal{L}(\tilde{\mathbf{w}}_{t+1})|\tilde{\mathbf{w}}_t] \leq (1 - c_1\eta + c_2\eta^2)\mathcal{L}(\tilde{\mathbf{w}}_t).$$

So if η is small enough, $1 - c_1\eta + c_2\eta^2 < 1$, and exponential convergence follows.

Problem 6

a

To compute gradients for layer 9, we need activations of layer 9 and 10 (layer 10 is stored). For layer 9, we compute 6 through 9 (4 fwd ops). For layer 8, we compute 5 through 8 (4 fwd ops), and so on until layer 6, which only needs 1 fwd op because we already have layer 5 stored. So total fwd operations: $4 + 4 + 3 + 2 + 1 + 1 = 15$.

b

For each layer from 6 to 10, we need the activation of layer 5 for recomputation. So we fetch layer 5's activation 5 times, and layer 10 once during the loss computation. Thus, 6 such operations are invoked.

c

Let the time taken for each `fwd` = 20ms and each `load_mem` = 10ms. Tensor rematerialization uses 15 `fwd` and 6 `load_mem`, so total time = $15 \cdot 20 + 6 \cdot 10 = 300 + 60 = 360$ ms.

We now replace each `load_mem` with a `load_disk` operation, and we want the total time to remain 360ms. Let x be the time (in ms) for a `load_disk`. Then,

$$15 \cdot 20 + 6x = 360 \Rightarrow 300 + 6x = 360 \Rightarrow x = \frac{60}{6} = \boxed{10 \text{ ms}}$$

Problem 7

a

Intermediate values:

$$\begin{aligned}
p &= u_1 \\
q &= w \cdot u_1 \\
r &= u_2 + q = u_2 + w \cdot u_1 \\
s &= w \cdot r = w \cdot (u_2 + w \cdot u_1) = w \cdot u_2 + w^2 \cdot u_1 \\
t &= u_3 + s = u_3 + w \cdot u_2 + w^2 \cdot u_1 \\
y &= w \cdot t = w \cdot u_3 + w^2 \cdot u_2 + w^3 \cdot u_1
\end{aligned}$$

b

Using the expression for y from above:

$$\frac{dy}{dw} = \frac{d}{dw}(w \cdot u_3 + w^2 \cdot u_2 + w^3 \cdot u_1) = u_3 + 2w \cdot u_2 + 3w^2 \cdot u_1$$

c

Partial derivatives:

$$\frac{\partial y}{\partial t} = w, \quad \frac{\partial y}{\partial s} = w, \quad \frac{\partial y}{\partial r} = \frac{\partial y}{\partial s} \cdot \frac{\partial s}{\partial r} = w \cdot w = w^2$$

$$\frac{\partial y}{\partial q} = \frac{\partial y}{\partial r} \cdot \frac{\partial r}{\partial q} = w^2 \cdot 1 = w^2, \quad \frac{\partial y}{\partial p} = \frac{\partial y}{\partial q} \cdot \frac{\partial q}{\partial p} = w^2 \cdot w = w^3$$

d

Use the derivatives from (c) to compute the partials along each edge from w:

- Edge from w to q:

$$\frac{\partial y}{\partial w}|_{w \to q} = \frac{\partial y}{\partial p} \cdot \frac{\partial p}{\partial w} = w^3 \cdot 0 + w^2 \cdot u_1 = w^2 u_1$$

- Edge from w to s:

$$\frac{\partial y}{\partial w}|_{w \to s} = \frac{\partial y}{\partial r} \cdot \frac{\partial r}{\partial w} = w^2 \cdot u_1 + w \cdot u_2$$

- Edge from w to y:

$$\frac{\partial y}{\partial w}|_{w \to y} = t$$

Total derivative:

$$\frac{dy}{dw} = \frac{\partial y}{\partial w}|_{w \to y} + \frac{\partial y}{\partial w}|_{w \to s} + \frac{\partial y}{\partial w}|_{w \to q} = t + (w \cdot u_2 + w^2 \cdot u_1) + w^2 \cdot u_1 = w \cdot u_3 + 2w^2 \cdot u_1 + w \cdot u_2$$

Which matches earlier result from part b.